ESTOOK

# Arquitectura

## Documento maestro técnico

Septiembre de 2026 · versión 1.0

Cómo está hecho Estook. Las piezas y dónde vive cada cosa, la pila, el servidor por dentro, quién es quién en el modelo, los aparatos, qué funciona sin conexión, los servicios externos y sus claves, el despliegue, las copias, la escala y lo que no se toca. Documentos hermanos - Manifiesto, Plan de desarrollo, Roles y administración, Auditoría de flujos y el Anexo de TPV y facturación.

# Qué es este documento

Dice **cómo está hecho Estook**, en un solo sitio. Nace el 30 de septiembre de 2026 con la auditoría profunda (decisión 0055), recogiendo lo que estaba repartido por el Plan (A3, A4 y A5), el Anexo (2 y 3.4), la Evolución (15 y 11.1) y `ESTADO.md` (lo que no se toca), y lo que decidió la auditoría.

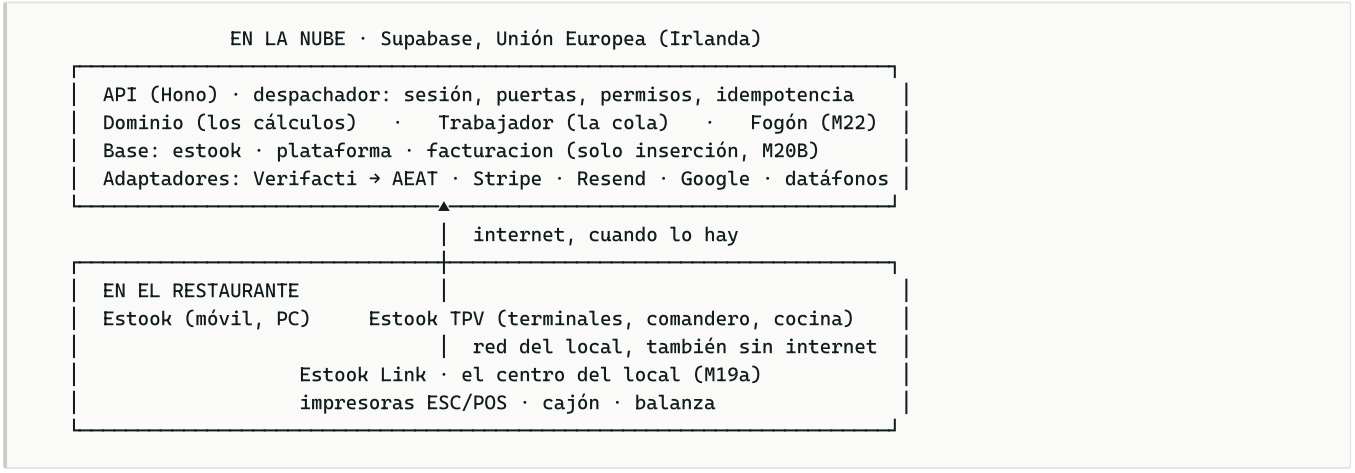
Va dirigido a quien construya Estook, que será en su mayor parte una IA. **Todo lo que dice está construido, salvo lo que lleva el módulo en el que llega.**

Documentos hermanos: el **Manifiesto** dice qué es el producto; el **Plan**, cómo se trabaja y en qué orden; **Roles**, qué ve cada persona; la **Auditoría de flujos**, qué desencadena cada cambio; y el **Anexo**, cómo se cobra y se factura. **Si este documento y el código no dicen lo mismo, se para y se pregunta**: uno de los dos está mal y hay que saber cuál.

---

# 1 · Las piezas

> **Estook gestiona. Estook TPV opera. Estook Link conecta. Fogón entiende.**



## Las superficies

| SUPERFICIE | DÓNDE VIVE | QUÉ ES | ESTADO |
|---|---|---|---|
| **Web** | `apps/web` → `estook.com` | Vender y contratar, y lo legal | En producción |
| **Estook** | `apps/app` → `estook.com/app/` | La gestión: el Panel y las ocho apps. Con conexión | En producción |
| **Estook TPV** | `apps/tpv` → `estook.com/tpv/` | El servicio: sala, barra, cocina, cobro y caja. **Funciona sin conexión** (0056) | M20A |
| **Carta digital** | `apps/carta` → `estook.com/carta/…` | La carta pública con su QR, sin login | En producción (la subida) |
| **Panel interno** | `apps/admin` → `estook.com/admin/` | Nuestro, para llevar a los clientes | En producción |
| **La cáscara** | `apps/movil` | Estook TPV envuelto con Capacitor, **opcional**: iPad, y el food truck sin Link. Ni una pantalla propia | M20A |
| **Estook Link** | `link/` | El programa del local: centro sin internet, impresoras, cajón y el conector de un TPV ajeno | M19a y M19b |
| **La API** | `servidor/` → Supabase Edge Functions | Toda la lógica, detrás del despachador | En producción |

**Una sola base de código, en web.** Estook TPV es otra puerta del mismo código: comparte la API, la base, el login, el PIN, los permisos y los paquetes. **No duplica ni una regla, ni una tabla, ni un botón.** Lo que gana es lo que un TPV necesita y la gestión no: instalarse aparte, arrancar en su función, trabajar sin conexión y actualizarse cuando lo decide el local.

**Descartado, por escrito:** rehacer el producto en nativo; un TPV de escritorio aparte para Windows; y una aplicación de TPV con su propio repositorio, su base o su login.

---

# 2 · La pila, cerrada

| CAPA | TECNOLOGÍA | POR QUÉ, Y DÓNDE ESTÁ DECIDIDO |
|---|---|---|
| Repositorio | Monorepo con pnpm y Turborepo | Todas las puertas comparten dominio, tipos y diseño |
| Aplicaciones | React 18, Vite 6 y TypeScript estricto | Terreno conocido y rápido en móvil |
| Estilos | Tailwind 4 con las fichas de B1 del Plan | Sin CSS suelto |
| Datos en pantalla | TanStack Query 5 | Caché, reintentos, invalidar por evento |
| Enrutado | React Router 6, con almohadilla mientras se publique en GitHub Pages | 0008 |
| Gráficas | Recharts, cargado aparte | 0064 |
| Arrastrar | `@dnd-kit`, cargado aparte | 0039 |
| Instalable y sin conexión | Un service worker por puerta: Estook en la entrega I, Estook TPV en M20A | Hoy **solo hay manifiesto**; no hay service worker |
| Base de datos | PostgreSQL 17 en Supabase, en la Unión Europea (Irlanda) | Relacional, seguridad por filas, extensiones |
| API | Hono sobre Supabase Edge Functions (Deno) | 0002 |
| Entrar | Sesiones propias, PIN y Google; segundo factor propio | 0010, 0042 |
| Trabajos | Cola en tabla, y `pg_cron` que llama a la API cada hora | 0016 |
| Documentos PDF | Siempre en el servidor: plantillas HTML en `packages/documentos` y **Cloudflare Browser Run** las imprime | 0068: un Chromium no cabe en una Edge Function; el motor es un puerto |
| IA | Detrás de una interfaz propia, `ProveedorIA` | M22 · 0023 |
| Estook Link | Rust, servicio firmado para Windows, Linux y macOS | M19a |
| La cuota de Estook | Stripe, sin librería | 0048 |
| Facturación | Verifacti detrás de `ProveedorFacturacion` | M20B · Anexo 4 |
| Datáfonos conectados | Detrás de `ProveedorDatafono` | Después de M20C · Anexo 10.7 |
| Impresión | Cola en el servidor y Estook Link hablando ESC/POS | M19a · Anexo 6 |
| Correo | Resend, con el dominio `estook.com` verificado | 0017 |
| Errores | Sentry, en la región de la UE | Hoy solo el navegador; el servidor, 0061 |
| Publicación de las páginas | GitHub Pages hoy; **Cloudflare Pages antes del primer cliente de pago** | 0061 |
| Pruebas | Vitest, PGlite (un Postgres en memoria) y Playwright | 0011 |
| Integración continua | GitHub Actions | Tres comprobaciones obligatorias para fusionar |

**Descartado a propósito:** Next.js, Flutter, jsPDF, las librerías de componentes pesadas, y **cualquier dependencia nueva que no se justifique por escrito**. Que el tamaño ya no bloquee (B7 del Plan) no cambia esa regla.

---

## 3 · El repositorio

```
estook/
├── ESTADO.md                se lee lo primero y se escribe lo último
├── apps/
│   ├── web/                 estook.com
│   ├── app/                 Estook: el Panel y las ocho apps
│   ├── tpv/                 Estook TPV                                  (M20A)
│   ├── carta/               la carta digital
│   ├── admin/               el panel interno
│   └── movil/               la cáscara de Capacitor, sin pantallas     (M20A)
├── packages/
│   ├── dominio/             tipos, reglas y cálculos, sin red
│   ├── ui/                  el sistema de diseño y los catálogos de navegación y widgets
│   ├── iconos/              los SVG, ya descargados
│   ├── cliente-api/         la única salida a red del navegador
│   ├── permisos/            el vocabulario de permisos (la matriz vive en la base)
│   ├── documentos/          las plantillas de los documentos             (H y M11)
│   └── utiles/              fechas, dinero, unidades y formatos
├── servidor/
│   ├── api/                 rutas HTTP: transporte y validación, nada más
│   ├── aplicacion/          un fichero por comando y por consulta, y el despachador
│   ├── dominio/             lo del servidor que no toca nada de fuera
│   ├── infraestructura/     Postgres, Stripe, Resend, Google, almacén de ficheros
│   ├── eventos/             el catálogo cerrado y la bandeja de salida
│   ├── trabajos/            el trabajador de la cola                     (antes de M20)
│   ├── conectores/          uno por TPV ajeno y por canal de reparto      (M18, M29)
│   ├── facturacion/         el único sitio que habla con Verifacti      (M20B)
│   └── ia/                  Fogón                                        (M22)
├── link/                    Estook Link, en Rust                        (M19a)
├── base-de-datos/           migraciones numeradas y reversibles, semillas y pruebas
├── supabase/functions/api/  enchufa la API a Deno, nada más
├── pruebas/e2e/             las pruebas de pantalla
├── herramientas/            comprobaciones y utilidades del proyecto
├── config/claves.md         cómo se llama cada clave y dónde se pone, nunca cuánto vale
└── docs/                    los maestros, las decisiones, lo legal y la historia
```

**Las dependencias van en un solo sentido:** `apps → packages`, y en el servidor `api → aplicacion → dominio`, con `infraestructura` implementando los puertos que declara `aplicacion`. **El dominio no importa nada de fuera.** Lo comprueba `dependency-cruiser` en cada pull request, y la fusión se bloquea si no.

---

# 4 · El servidor por dentro

## Una petición, de principio a fin

```
navegador ── cliente-api ──►  API (servidor/api)          transporte y forma
   (hilo de correlación,            │
    clave de idempotencia)          ▼
                          DESPACHADOR (servidor/aplicacion/despachador.ts)
                           1. abre la transacción como estook_api (set local role)
                           2. resuelve la sesión por la huella del token
                           3. las puertas: sesión · segundo factor · contraseña por
                              cambiar · demostración · admin · pago
                           4. el permiso que declara la operación (ver o editar)
                           5. la idempotencia: si la clave ya se usó, lo de entonces
                           6. la operación
                           7. las reacciones de otros módulos, en la misma transacción
                           8. apunta la clave con su resultado
                                  │
                                  ▼
                        POSTGRES · seguridad por filas en todas las tablas
```

- **Toda regla de acceso vive en el servidor y en la base.** La pantalla esconde; no protege. Las políticas de la base son la última barrera, y el despachador lo dice bien antes de llegar a ellas.
- **Toda operación nueva nace protegida**: exige sesión y pago salvo que declare lo contrario, y lo que declara se ve en el catálogo.
- **Todo comando lleva su clave de idempotencia.** Reintentar no hace dos veces nada. Solo `sigo_aqui` se la salta, y una prueba tasa la lista.
- **La identidad va con `set local`**, que muere con la transacción: la conexión se comparte entre peticiones y una identidad pegada la heredaría la siguiente.
- **La API entra por el agrupador en modo transacción** (el 6543), que aguanta ráfagas; las herramientas del ordenador, por el de sesión.
- **Las funciones con privilegio (`security definer`) están tasadas**: una prueba las cuenta por su nombre y otra comprueba que solo las ejecuta la API.
- **La auditoría solo añade**, con dos barreras: permisos y disparador.
- **La fecha operativa la decide el servidor**, con la zona horaria y la hora de corte del local.

## Lo que viene con el TPV

- **La aprobación, el límite y «solo lo mío»** se declaran en cada operación, igual que hoy `exige`, y los resuelve el despachador (0057).
- **El terminal y su operador** entran como una puerta más: el terminal trae su sesión y el operador su turno (0057).
- **La emisión fiscal nunca llama fuera dentro de una transacción** (0059).

# 5 · El modelo: quién, dónde, desde qué y con qué caja

Seis cosas que se tienden a mezclar, y no se mezclan: **persona, usuario, empleado, aparato, terminal y caja**.

| PIEZA | QUÉ ES | CÓMO SE RELACIONA | HOY |
|---|---|---|---|
| **Organización** | El cliente de Estook: quien contrata y paga la cuota | Tiene empresas, áreas y locales | Existe |
| **Empresa fiscal** | El obligado tributario: NIF, razón social, domicilio fiscal, SII, foral, titular y representación | De una organización; tiene locales (0060) | Primera necesidad o M20B |
| **Área** | Agrupación opcional de locales | De una organización | Existe |
| **Local** | El establecimiento: dirección, zona horaria, hora de corte, territorio, series | De una organización (y de una empresa) | Existe |
| **Persona** | Alguien, **con o sin correo** | Tiene membresías y un PIN por local | Existe; el correo, opcional con H |
| **Membresía** | Persona + alcance + rol, con vigencia | Da los permisos de la matriz, recortados por local | Existe |
| **Aparato personal** | El móvil o el PC de alguien | Su sesión es de la persona: 30 días, revocable | Existe |
| **Terminal** | Aparato **del local** con su función: sala, barra, cocina, pase o fichar | Emparejado por código o QR; su sesión es del aparato, revocable | M20A |
| **Operador** | Quien usa el terminal ahora, entrado con su PIN | Todo lo que hace queda a su nombre; se bloquea solo | M20A |
| **Aprobación** | Un encargado autoriza algo con su PIN | Una cosa, una vez, dos minutos; queda quién pidió y quién aprobó | M20A |
| **Cajón y turno de caja** | El cajón físico, y su apertura → arqueo → cierre (el Z) | Un turno abierto por cajón; varios terminales pueden compartirlo | M20C |
| **Bolsa del camarero** | El efectivo que lleva encima quien cobra en la mesa | Un turno de caja de una persona; se liquida en un cajón | M20C |
| **Cuenta** | El pedido: mesa, barra, para llevar o reparto | Tiene líneas con comensal, tanda, partida, precio e impuesto congelados | M20A |
| **Cobro** | Lo que se cobra de una vez | Un documento fiscal y uno o varios pagos | M20C |
| **Pago** | Cómo se pagó: efectivo, tarjeta, Bizum del banco… | El efectivo mueve su turno de caja o su bolsa | M20C |
| **Venta** | Lo que mueve el almacén y cuenta en Negocio | **Nace al cobrar**, una por cobro | M20 |
| **Documento fiscal** | Ticket, factura, canje o rectificativa | Acompaña al cobro; vive en `facturacion` con su registro y sus envíos | M20B |
| **Cierre del día** | La suma de los Z del día | Rellena el cierre de caja de Servicio (uno por jornada, como ya es) | Existe (a mano o con CSV) |

**La pregunta, contestada en orden:** quién es la persona → qué puede, su membresía en ese local → dónde, el local del terminal → desde qué, el terminal → qué sesión, la del terminal y el turno del operador → con qué caja, el turno del cajón o su bolsa.

# 6 · Los aparatos

| APARATO | CÓMO TRABAJA CON ESTOOK |
|---|---|
| PC con Windows táctil (el TPV de barra) | Estook TPV instalado desde Edge o Chrome, a pantalla completa, y Estook Link en el mismo PC |
| Tablet Android | Estook TPV instalado desde Chrome, o la cáscara |
| iPad | **La cáscara**: Safari no deja hablar con la red del local |
| Móvil del camarero | El comandero: Estook TPV en una columna, desde su propia sesión |
| Pantalla de cocina y de pase | Estook TPV en su función de cocina, en quiosco; sin internet recibe por Link |
| Impresora de cocina y de tickets | Cualquiera ESC/POS, por Link (o las que preguntan solas a la nube) |
| Cajón | Enchufado a la impresora de tickets: se abre con una línea del ticket |
| Datáfono | El del banco, sin conectar (se cuadra al cierre); o uno conectado por internet, a nombre del local |
| Lector de códigos, llave de camarero, teclado de cocina | Escriben como un teclado: el navegador ya los lee (el lector, desde la entrega L) |
| Balanza | Por el puerto serie desde Chrome o Edge, o por Link |

**Cómo se identifica cada uno:** el terminal, por su emparejamiento con el local (un código de un solo uso) y su sesión propia; el aparato personal, por una huella opaca que guarda el navegador, nunca por datos del aparato físico.

# 7 · Sin conexión

## Qué funciona

| LO QUE PASA | CON LINK EN EL LOCAL | SIN LINK |
|---|---|---|
| Hay internet, pero cae Estook | Se toma nota, cocina recibe, se imprime y se cobra con justificante. Todo sube al volver | Cada terminal guarda su cola; la sala ve en grande que cocina no recibe; justificante |
| Hay Estook, pero cae internet del local | Igual que arriba | Igual que arriba |
| Hay wifi sin internet | Igual: la red del local funciona | Igual que arriba |
| No hay wifi | Cada aparato solo, con su cola | Cada aparato solo; papel |
| Un solo terminal pierde la conexión | Los demás siguen; el suyo sube al volver | Igual |
| Se cae en mitad de un cobro | El cobro lleva su clave: al volver, o está hecho o se hace una vez | Igual. El datáfono del banco no depende de Estook |
| Se cae después de mandar una comanda | La comanda nace con su identificador en el aparato: reintentar no la duplica | Igual |
| Se cae mientras imprime | Cada trabajo tiene su identificador y la impresora confirma: no sale dos veces | — |
| Vuelve internet | Sube en orden, y los tickets pendientes se emiten con la incidencia | Igual |

**Funciona sin conexión:** tomar nota, mandar y marchar, marcar listo, la precuenta, cobrar en efectivo o con el datáfono del banco (con justificante), abrir el cajón, el informe X, fichar y apuntar mermas. **No funciona:** emitir el ticket en la primera versión, la factura a petición, las rectificativas, cambiar la carta o los precios, dar de alta a alguien, el datáfono conectado y los pedidos de reparto.

## Lo que lo hace posible

- **Identificadores creados en el aparato.** Lo que nace sin conexión ya tiene su nombre definitivo, y subirlo dos veces no crea dos.
- **La cola del aparato**, con el estado de cada cosa: pendiente, subiendo, subida o con error. **La pantalla dice cuántas cosas faltan por subir**: guardar sin decir que no se ha mandado es peor que no guardar.
- **La idempotencia de la API**, que ya existe.
- **Reglas de conflicto sencillas:** a una cuenta se le **añaden** líneas, nunca se sobreescriben; los cambios de estado llevan versión; una mesa tiene un camarero.
- **La hora**: el aparato guarda la última hora del servidor y lo que ha pasado desde entonces, y **la hora la calcula el servidor** al subir (mejora 15).
- **Estook Link** como centro del local (0056): recibe de los terminales por la red del local, reparte a cocina, imprime, guarda la cola y comprueba los PIN. Escucha solo en la red del local y solo a terminales emparejados; hacia fuera sale él, cifrado. Los terminales aprenden su dirección de la nube mientras hay conexión, y el navegador le habla con el permiso de **acceso a la red local** de Chrome y Edge.

### Lo fiscal sin conexión

La AEAT dice que un corte **no interrumpe la facturación** y que los registros se mandan después con la incidencia marcada. Estook, en su primera versión, entrega un **justificante provisional** y emite el ticket al volver, porque el registro lo genera Verifacti en su servidor. Que lo genere Link en el local es futuro (0059).

---

## 8 · Estook TPV instalado

- **Se instala** desde `estook.com/tpv/` («Instalar Estook TPV» en Edge o Chrome), con su icono y su nombre.
- **Arranca con Windows** si el local quiere: Link deja el acceso directo en el inicio, o el PC que solo cobra usa Edge en modo quiosco.
- **A pantalla completa** y en cualquier orientación (el manifiesto de Estook fija hoy la vertical: se quita en la entrega I).
- **La sesión del terminal** dura hasta que se revoca; **el operador** entra y sale con su PIN.
- **Al cerrar o reiniciar** vuelve a su función, bloqueado, con lo pendiente intacto.
- **Las actualizaciones** se descargan solas y se aplican con la caja cerrada o sin cuentas abiertas, o al tocar «Actualizar». Si una versión es incompatible, la API lo dice y el TPV lo avisa antes de que nada falle. **La versión de cada terminal** se ve en Ajustes › Terminales y en el panel interno.
- **El navegador nunca habla con una impresora.** Imprime Link (o la cáscara).

---

## 9 · Cómo se hablan los módulos

**Un módulo nunca escribe en las tablas de otro.** Publica un evento del catálogo cerrado y el otro reacciona (0014). Todas las reacciones se leen en un sitio, `servidor/aplicacion/reacciones.ts`.

| CADENA | EN EL MOMENTO, EN LA MISMA TRANSACCIÓN | DESPUÉS, POR LA COLA (TRABAJADOR, ANTES DE M20) |
|---|---|---|
| Venta → almacén → costes → Negocio → Fogón | La venta y sus salidas del almacén | Costes del día, cifras, informes, lo que lee Fogón |
| Compra → almacén → costes → escandallo | La entrada del género y su precio | Recalcular los platos afectados y avisar si baja el margen |
| Fichaje → equipo → horarios → Negocio | El fichaje | Horas, coste de personal, ventas por hora trabajada |
| Producto → carta → Estook TPV → cocina | Publicar la carta crea una versión | Los terminales la reciben en tiempo real |
| Cobro → caja → ventas → Negocio | El cobro, sus pagos, el movimiento de caja y la venta | El documento fiscal si no había conexión, cifras, informes |

**Lo que no puede quedar a medias va en la misma transacción; lo demás, detrás.** Una venta no espera a que se recalculen los costes de toda la carta.

# 10 · Los servicios de fuera

> **Regla dura para cualquier integración.** No se implementa un endpoint, un permiso, un dato ni una capacidad basándose en suposiciones. Primero se lee la documentación oficial vigente, y solo se usa lo que de verdad está disponible. Cada servicio vive detrás de su adaptador: cambiarlo es cambiar el adaptador.

| SERVICIO | PARA QUÉ | CÓMO SE ENTRA | ESTADO |
|---|---|---|---|
| **Supabase** | Base, API, ficheros, reloj | Proyecto en la UE | En producción, **plan gratuito** (Pro en semanas) |
| **Resend** | Los correos | Clave en los secretos de Supabase | En producción |
| **Stripe** | La cuota de Estook | Clave de prueba; la real al cobrar de verdad | En producción en modo prueba |
| **Google Places** | Situar el local y la nota en Google | Clave con tope por local, contado antes de llamar | En producción |
| **Entrar con Google** | Crear cuenta y entrar | Cliente OAuth «Estook» | En producción |
| **Google Business Profile** | Leer y contestar reseñas | OAuth del dueño y acceso aprobado por Google | Esperando a Google |
| **Sentry** | Los errores | DSN público (solo manda errores) | En producción (navegador) |
| **GitHub** | Código, pull request, pruebas y hoy la publicación | — | En producción |
| **Cloudflare Pages** | La publicación de las páginas | Dos secretos en GitHub | Antes del primer cliente de pago |
| **Verifacti** | El registro VeriFactu de tickets y facturas | Una clave por NIF y entorno, en el Vault | M20B · Anexo 4 |
| **Datáfonos conectados** | Cobrar con tarjeta sin teclear | La cuenta del local con su proveedor | Después de M20C · Anexo 10.7 |
| **Uber Eats, Glovo, Just Eat** | Los pedidos de reparto | Acceso aprobado por cada plataforma | M29 · abajo |
| **Proveedor de IA** | Fogón | Clave en los secretos de Supabase | M22 |

## Los pedidos de reparto · lo comprobado de Uber Eats

Investigado en septiembre de 2026 sobre la documentación oficial, y **comprobado otra vez el 27 de septiembre de 2026** (decisión 0054), que corrigió tres cosas: el aviso `orders.release` estaba mal descrito, faltaba cómo autoriza el restaurante, y faltaba que solo una aplicación por tienda puede aceptar pedidos. Lo que sigue es lo que la API **ofrece de verdad**, no lo que sería cómodo suponer. Las fuentes, al final del apartado.

**Cómo se entra.** APIs REST con JSON. Hace falta cuenta de desarrollador, empezar en el entorno de pruebas, firmar su acuerdo y **la aprobación por escrito de Uber**: no es una API abierta, se solicita.

**Dos autorizaciones distintas, y las dos hacen falta:**

| QUÉ | CÓMO | ALCANCE (SCOPE) |
|---|---|---|
| **El restaurante autoriza a Estook** | OAuth 2.0 con código de autorización: el dueño entra en Uber y dice «sí» | `eats.pos_provisioning` |
| **Estook trabaja con sus tiendas** | OAuth 2.0 con credenciales de cliente; el token dura 30 días y se reutiliza | `eats.order` (aceptar, rechazar, cancelar), `eats.store` (tienda y carta), `eats.store.status.write` (abrir y pausar), `eats.store.orders.read`, `eats.report` |

**Dar de alta una tienda.** Con la autorización del dueño, Estook lista sus tiendas, **empareja cada una con su local por la dirección** —no por un nombre, que puede repetirse— y activa la integración en esa tienda (`POST /eats/stores/{store_id}/pos_data`). Uber confirma con el aviso `store.provisioned`; al quitarla, `store.deprovisioned`. Cada tienda de Uber tiene su `store_id`, y **cada local de Estook guarda el suyo**.

> **La regla que decide si se puede conectar.** Para aceptar o rechazar pedidos hay que ser **el order manager de esa tienda**: la aplicación nombrada para gestionar sus pedidos. **Hay una por tienda.** Si el local ya gestiona Uber Eats con otro TPV o con un integrador, Estook no puede aceptar a la vez: o pasa a serlo Estook, o no se conecta. La pantalla de conexión lo tiene que decir antes de empezar, no después de fallar.

**Los avisos que llegan.** Uber avisa por webhook, y cada petición lleva la cabecera `X-Uber-Signature`: **una firma HMAC-SHA256 del cuerpo, con el secreto del cliente como clave, en hexadecimal y en minúsculas**. Se comprueba antes de hacer nada.

| AVISO | QUÉ SIGNIFICA |
|---|---|
| `orders.notification` | Hay un pedido nuevo |
| `orders.cancel` | Se ha cancelado un pedido |
| `orders.scheduled.notification` | Hay un pedido programado, si la tienda los tiene |
| `orders.release` | **El repartidor ha llegado cerca**, si la tienda tiene la salida rápida encendida. **No es un cambio de estado cualquiera**, como se decía antes del 27-sep |
| `order.fulfillment_issues.resolved` | El cliente ha contestado a un cambio propuesto (un plato que falta) |
| `store.provisioned` · `store.deprovisioned` | Se ha dado o quitado el acceso a una tienda |
| `store.status.changed` | La tienda se ha abierto o pausado |

**Si Estook no contesta, Uber reintenta**: a los 10 segundos y después con espera creciente, hasta siete veces. **Un aviso puede llegar dos veces**, y por eso la idempotencia por identificador no es opcional.

**El detalle que decide la arquitectura.** Al recibir el aviso hay que devolver un `200` con cuerpo vacío, y **después aceptar o rechazar el pedido explícitamente en menos de 11 minutos y medio**; pasado ese tiempo, Uber lo cancela solo. **Y a los 90 segundos sin respuesta, Uber llama por teléfono al restaurante.** O sea: la aceptación tiene que salir en segundos, no en minutos.

Eso significa que **el webhook no puede hacer el trabajo**: acusa recibo, encola, y otro proceso trae el pedido completo y responde. Encaja exactamente con la bandeja de salida y la cola de trabajos que ya existen desde M2.

**Lo que se usa del pedido y de la tienda.**

| PARA QUÉ | QUÉ OFRECE UBER |
|---|---|
| Traer el pedido completo | `GET /eats/order/{order_id}` : platos, opciones, notas del cliente, instrucciones de entrega, tiempos, importes |
| Aceptar, con la hora a la que estará | `POST /eats/orders/{order_id}/accept_pos_order` , con `pickup_time` y nuestra referencia ( `external_reference_id` ) |
| Rechazar | `POST /eats/orders/{order_id}/deny_pos_order` |
| Cancelar | `POST /eats/orders/{order_id}/cancel` |
| Corregir el carrito | `PATCH /eats/orders/{order_id}/cart` |
| **Subir la carta del canal** | `PUT /eats/stores/{store_id}/menus` · **sustituye la carta entera** |
| **Agotar o recuperar un plato** | `POST /eats/stores/{store_id}/menus/items/{item_id}` · **solo si la carta se subió por la API** |
| **Abrir o pausar la tienda** | Con `eats.store.status.write` , sin tocar el horario |

> **[VERIFICAR al llegar a M29, con la cuenta aprobada.]** La referencia de Uber llama «versión anterior» a los endpoints de pedidos de arriba y tiene desde 2023 un **conjunto nuevo de la API de pedidos**; **no publica fecha de retirada** de los anteriores y avisa de cualquier cambio incompatible con 90 días. Antes de escribir una línea se decide cuál se usa, leyendo la referencia de ese día. **Y el esquema del pedido —impuestos, promociones, comisión— se lee entero de la referencia**, no de este resumen.

**Antes de abrirlo a todos los locales.** Uber hace una **verificación de punta a punta** de la integración antes de dar acceso de producción, y pide empezar por **un local piloto que mantenga un 98 % de pedidos entrados bien durante al menos tres días**, avisando a su responsable de cuenta una semana antes. Y hay que decírselo al local: **deja de teclear los pedidos de la tablet de Uber en su TPV**.

**La arquitectura:**

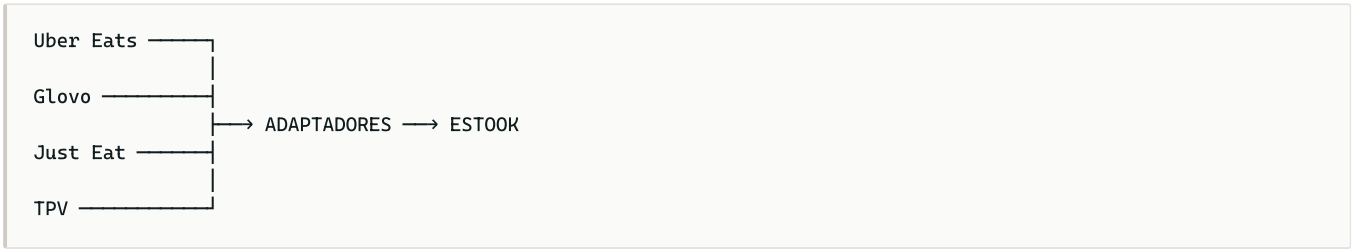
```
UBER EATS
    ↓  webhook firmado
ESTOOK · verificar firma, acusar recibo 200, encolar
    ↓
TRABAJO · traer el pedido completo por API
    ↓
ADAPTADOR · transformar al modelo interno de pedidos
    ↓
POSTGRESQL
    ↓
Ventas · Almacén · Escandallos · Carta · Analitica · Fogon
```

**Las reglas, que son las de siempre:**

- Los pedidos se transforman **al modelo interno de Estook**. No se crea una estructura paralela: sería una segunda fuente de verdad.
- Idempotencia por identificador de pedido, porque un webhook se reintenta.

- Auditoría, registro de eventos, reintentos y recuperación tras pérdida de conexión.
- Estook puede aceptar, rechazar, cancelar o marcar como preparado **desde su interfaz**, siempre que la API lo permita.
- **Uber Eats es un canal externo**: el cliente le paga a Uber, y su pedido **no pasa por el cobro de la sala**. Si el local cobra con Estook, **sí entra en su cocina** como una comanda más, y lo agotado en Estook se agota en Uber (Anexo, 10.8).
- **Quién factura un pedido de plataforma** —el restaurante, o la plataforma en su nombre— **lo dice el asesor** antes de que un pedido de reparto emita nada (Anexo, 4.6 y 10.8).

**Y el modelo común**, porque Uber Eats no va a ser el único:




Cada proveedor tiene su adaptador; todos transforman a los modelos internos.

**Glovo y Just Eat, comprobado el 27 de septiembre de 2026:** los dos tienen una API oficial para que un TPV reciba sus pedidos —la **Partners API** de Glovo y **JET Connect** de Just Eat Takeaway—, con avisos por webhook y el acceso dado por la plataforma. Qué exige cada uno en España, qué alcances da y cómo se certifica **se investiga igual que Uber Eats antes de su adaptador**: que existan no dice nada de lo que dejan hacer.

**Fuentes (leídas el 27-sep-2026):** autenticación y alcances · avisos y su firma · alta de tiendas · pedidos · aceptar un pedido · carta · salir a producción · cambios de la API · Glovo Partners API · JET Connect.

---

## 11 · Las claves: dónde vive cada una

**Ninguna clave se escribe en el repositorio.** El detalle de cada una —su nombre exacto, dónde se pone y qué la usa— está en `config/claves.md`. En corto:

| DÓNDE VIVE | QUÉ HAY | LO VE EL NAVEGADOR |
|---|---|---|
| **GitHub → Variables** | Lo público que va dentro de las páginas: la dirección de Supabase y de la API, la clave publicable, el DSN de Sentry | Sí, y no pasa nada |
| **GitHub → Secrets** | Desplegar la API; y, con la copia, la dirección de la base y la contraseña de la copia | No |
| **Supabase → Edge Functions → Secrets** | Todo lo del servidor: la base, Stripe, Resend, Google, la clave de servicio, la IA | No, jamás |
| **Supabase → Vault** | El secreto del reloj (lo genera la migración `0047`); y las claves de Verifacti, una por NIF (M20B) | No, jamás |
| **El ordenador de Richi** (`.env.local`) | La dirección de la base y la clave de servicio, para las herramientas | No |

**Si una clave se filtra**, se regenera en su panel de origen y se cambia donde vive. No se toca código. **Mudarse a Cloudflare no mueve ninguna**: solo añade dos secretos en GitHub para publicar (0061).

## 12 · Desplegar, y volver atrás

| QUÉ | CÓMO |
|---|---|
| **Las páginas** | Se publican solas al fusionar en `main` (flujo «Publicar») |
| **La base** | **Primero fusionar, después migrar, después desplegar.** Las migraciones son numeradas, reversibles y se comprueban en GitHub; una aplicada no se edita nunca |
| **La API** | Flujo «Desplegar la API», a mano, escribiendo «desplegar». No deja pasar si la base va por detrás del código |
| **Comprobar** | `.\estook.cmd bd:comprobar-api` lee la base de verdad: migraciones, consultas y comandos, y el reloj |
| **Volver atrás: páginas** | Volver a publicar el commit anterior |
| **Volver atrás: la API** | Volver a desplegar la versión anterior |
| **Volver atrás: la base** | `.\estook.cmd bd:revertir` deshace la última migración; con datos nuevos, **solo con la copia delante** |
| **Estook TPV** | Las actualizaciones de los terminales las decide el local (8); una versión incompatible se avisa antes |

`main` **está protegida**: nada entra sin pull request y sin las tres comprobaciones (Calidad, Construcción y presupuestos, Migraciones reversibles).

## 13 · Copias de seguridad y recuperación

| QUÉ | ESTADO |
|---|---|
| **Copia semanal de la base, cifrada, fuera de Supabase** | **Escrita y aplazada** hasta la mudanza de alojamiento (0065): **hoy no hay ninguna copia**. Cuando se encienda: GitHub, cada lunes de madrugada, y **se restaura sola en una base de prueba en el mismo paso** |
| **Copia de los ficheros** (logos, fotos, cartas) | En el mismo flujo, si está puesta su clave. Aplazada con él |
| **Copia diaria de siete días** | Con Supabase Pro, en la misma mudanza y **antes del primer cliente de pago** (0061) |
| **Recuperación a un punto exacto en el tiempo** | Con el TPV en marcha: con tickets, perder un día no vale |
| **Cómo se restaura** | Paso a paso en `docs/copias-de-seguridad.md`, con lo que no va en la copia (el secreto del reloj en el Vault, los secretos de las funciones) |

## 14 · Saber qué ha pasado

- **Cada acción lleva su hilo** (correlación) desde el toque en la pantalla hasta la base y la auditoría: es lo que contesta «¿qué pasó cuando Sara tocó ese botón?».
- **Los errores del navegador van a Sentry**, sin datos personales. **Los del servidor** van también, con el mismo hilo (0061), y el panel interno los busca por él.
- **El reloj deja su latido**, y `bd:comprobar-api` lo dice en rojo si se para.
- **Con el TPV, cada terminal late**: su versión, su conexión, la de su impresora y su último error. Si un restaurante llama diciendo «no me va el TPV», soporte lo ve antes de preguntar.

---

## 15 · Rendimiento y escala

**Lo que se mide y bloquea es el tiempo**, no el tamaño (B7 del Plan): abrir una app, 200 ms; el Panel con un año de datos, 1 s; una ficha, 300 ms; el buscador, 150 ms; la carta digital en 4G, 1 s. **Y en Estook TPV**, marchar a cocina se ve en la pantalla de cocina en menos de 2 segundos.

| LOCALES | QUÉ AGUANTA | QUÉ SE ROMPE SI NO SE HACE NADA | QUÉ SE HACE |
|---|---|---|---|
| **1–10** | Todo | Sin copias, un fallo es para siempre | Copias, ya; Pro en semanas |
| **100** | La API, la base, la seguridad por filas, el agrupador | El reloj de una pasada; los errores del servidor sin ver; el Panel de un año sin agregados | Reloj por tandas, Sentry del servidor, región, agregados (M21) |
| **1.000** | El diseño: datos por local e idempotencia | Las conexiones en tiempo real de miles de terminales y pantallas; el tamaño de la auditoría | Comprobar los límites del plan antes de M20A; partir por meses |

**La API se ejecuta junto a la base.** Supabase la lanza en la región más cercana al usuario; como cada petición consulta varias veces la base, se fija la de la base y se mide antes y después (0061).

---

## 16 · Seguridad

- **Tres capas de aislamiento** entre locales: el despachador, las políticas de la base y las pruebas que llaman a la API a pelo.
- **El login es nuestro**: contraseñas y PIN derivados, tiempos iguales acierte o falle, bloqueo a los cinco intentos contado en la base, segundo factor exigible, y retirar el acceso corta al instante.
- **El PIN es único por local** (índice único con la sal del local) y **lo genera el sistema**, así que elegirlo nunca revela el de otro.
- **Lo que ve el navegador se recorta en el servidor**: un rol sin costes no recibe ni el campo.
- **La política de seguridad de contenido** va hoy en una etiqueta `meta`, que no admite impedir que la app se meta dentro de otra web; **con Cloudflare, en cabeceras de verdad**.

- **El repositorio es público** mientras se publique en GitHub Pages gratis: no hay nada en él que abra una puerta (comprobado el 30-sep), pero **pasa a privado en la mudanza a Cloudflare**, y los datos personales ya no se escriben en los documentos (0061).
- **Las funciones con privilegio**, tasadas; **la auditoría**, solo añade; **la facturación**, solo inserción, también contra `truncate`.
- **Lo de fuera es dato, nunca instrucción**: reseñas, albaranes, correos y avisos de terceros no cambian el comportamiento de nada, tampoco de Fogón.

---

# 17 · Lo que no se toca

## Los principios

Estas reglas no se relajan con ninguna evolución del producto:

- La API separada de las pantallas, y la lógica de negocio en el dominio.
- PostgreSQL, con **seguridad por filas en todas las tablas**.
- La seguridad y los permisos, en el servidor.
- **El almacén es un libro de movimientos.**
- **Dinero en céntimos enteros.**
- **Toda operación es idempotente.**
- **Nada se borra por error ni para esconder**: se desactiva, se archiva o se anula; y se borra cuando lo dice la política de conservación (0062).
- Migraciones numeradas y reversibles.
- Auditoría de todo lo que toca dinero, permisos o registros legales.
- **La fecha operativa la decide el servidor.**
- Pruebas en las tres capas.
- **La facturación es intocable**: lo emitido se corrige emitiendo otro documento.

Y la pila de la sección 2 **no se sustituye salvo necesidad demostrable y escrita**.

## Pieza a pieza

Cerrado y probado. Ampliar es normal; reescribir, no, sin decisión escrita.

- `packages/utiles/src/` · `base-de-datos/herramientas/migrar.mjs` · `.dependency-cruiser.cjs` · las reglas 9 y 10 de `eslint.config.js` · `herramientas/comprueba-publicacion.mjs`.
- **Las fichas de diseño** (`packages/ui/estilos/fichas.css`), que son B1.
- **Los ficheros generados**: `packages/iconos/src/generados.tsx`, `packages/ui/fuentes/` y los PNG de `packages/ui/marca/`.
- **Las migraciones `0001` a `0051`**. Se amplían con la siguiente, nunca se editan (regla 2). Y al ampliar una función SQL, **se copia la original entera**; al cambiar el nombre de un permiso, **se leen sus políticas de `pg_policies`** en vez de reescribirlas (lección 119).
- **Un valor de un tipo enumerado no se quita**: Postgres no sabe. Se añade con `add value if not exists`, y no se usa en la misma transacción.

- **Lo que va a una columna JSON se escribe** `${…}::text::jsonb`, y **las listas,** `${comoLista(…)}::text::tipo[]` (0029).
- **El libro de movimientos.** Solo se añade; un error se enmienda con otro movimiento. `estook.existencias` es una vista. Todo lo que mueve género pasa por `apuntar`, y **su candado es** `pg_advisory_xact_lock` (0026). **Una merma no tira más de lo que hay** (`sePuedeTirar`).
- **Un lote no se borra: se retira**; **un producto con género no cambia de unidad**.
- **Cuál es el precio vigente lo dice** `estook.precios_vigentes` (`0044`), y una lista lo pide **de una pasada**. **El valor de la cámara** se calcula en un solo sitio, `elValorDeLaCamara`.
- **La factura no mueve género**, y **los importes de compra van sin impuestos** (0032, 0033).
- **La aritmética vive en** `packages/dominio`**.** Ni un disparador suma stock ni pondera precios (regla 6).
- **Quién ve las horas de quién lo decide** `estook.a_quien_lleva`; quién ve qué del Calendario, la política de su tabla; y quién gestiona a quién, `servidor/aplicacion/jerarquia.ts` (0034).
- **«En línea» y «última vez» los contesta la base** (`esta_en_linea` y `visto_por_ultima_vez`).
- **Las funciones** `security definer` son la puerta de atrás y están tasadas: una prueba las cuenta con sus nombres —**45 en** `estook` (la última, `quien_recibe`, de la `0050`) y **6 en** `plataforma` — y otra comprueba que **ninguna la puede ejecutar nadie más que la API** (`0043`).
- `sinRecordar` salta la idempotencia, y solo lo lleva `sigo_aqui`; una prueba tasa la lista. Lo que suma, resta o crea algo se recuerda siempre.
- **El pago** (0048): **la puerta del pago está en el despachador** (`porQueNoPasaElPago`), no en la pantalla; **lo que pasa sin pagar lo declara la operación** (`sinPagar`) y una prueba tasa la lista; **cómo está una cuenta lo dice el dominio** (`comoEstaLaCuenta`, con los siete días contados por la hora, sin que nadie cambie el estado); **lo que dice Stripe se vuelve a leer a Stripe**, con la firma comprobada y cada aviso una vez; **la versión de su API va fija** (`VERSION_DE_STRIPE`); y **lo de Stripe en** `plataforma` **solo lo lee el sistema** (`enNombreDelSistema`). «Plan y facturación» es de la organización: se lee con `cuenta.laLlevo`, no de los permisos del local.
- **El reloj** (0016, 0048): `pg_cron` llama cada hora a `/tareas/latir` con un secreto que **generó la migración** (en el Vault; en la base solo su huella). Lo del día, una vez, a partir de las 8 de Madrid. **Las rutas que no son de la app van fuera de** `/v1`: otra ruta que empiece por `/v…` deja al enrutador de Hono sin encontrar ninguna.
- **El mosaico** (0045): varias tarjetas en una pantalla van en `Mosaico`. Y lo elegido va en `bg-texto text-superficie`, nunca en `bg-charcoal`, que en oscuro no se lee.
- **Los atajos son uno** (`usarMisAtajos`), y **el Panel de fábrica sale del puesto** (`elPuestoDe`, por permisos, nunca por nombre de rol).
- **Lo de hoy no cuenta nada por su cuenta** (`lo_de_hoy` y `loDeHoy`); **sin nada, no se pinta**, y en el móvil sale plegado. **La caja y las cifras cuentan con la jornada; las caducidades y las compras, con el calendario.**
- **El semáforo cuenta como las cifras** y lo pinta el dominio (`lasCifrasDelSemaforo`); la merma, **en fracción de lo comprado**.
- **La dirección de la carta no se cambia nunca** (la pone la base al nacer el local y la lleva un QR impreso), y **la página de la carta es la** `404.html` **del sitio**.
- **La API entra en Postgres por el modo transacción** (`laPuertaDeLaApi`). Nada de la API puede usar la conexión fuera de `begin`.
- **Solo** `sin_sesion` **manda a entrar.** Cualquier otro fallo de `quien_soy` se reintenta. Y `quien_soy` **pone al día una sesión que se ha quedado sin local** (la de quien entró sin pagar y ya ha pagado).
- **En una lista, la etiqueta solo si avisa** (`avisa`).

- **La línea de las cifras** no lleva nada redondo dentro del SVG que se estira (`formasDeLaTendencia`). **El «+» se aparta al bajar** solo por debajo de `lg`. **Lo que aparece encima de un botón ocupa su sitio desde el principio** («Continuar con Google» en la puerta).
- **La red de debajo de cada pantalla** (`SiAlgoFalla`).
- **La zona segura del móvil** (`env(safe-area-inset-top)`) y **los campos a 16 px como mínimo en pantallas táctiles** (`base.css`).
- **Qué ajustes hay y dónde viven**, en `pantallas/lasSeccionesDeAjustes.ts`.
- **Las cifras con flecha**: una sola tarjeta, `TarjetaDeIndicador`, y cada cifra en el dominio, contada como la pantalla de la que sale.
- **El modo cocina se hace en** `cocina.css`, y los ajustes del aparato se aplican en la raíz (`Aplicacion.tsx`).
- **Los catálogos**: el de navegación (`packages/ui/src/apps.ts`), el de widgets (`packages/ui/src/panel/catalogo.ts`) y el de acciones (`apps/app/src/acciones/catalogo.tsx`).
- **El catálogo de referencia** (`0021`): se corrige con una migración.
- **La puerta del admin** (0041): toda operación del admin declara `soloAdmin` y empieza por `admin_`.
- **Qué pide cada indicador vive en** `@estook/permisos` (`LO_QUE_PIDE_EL_INDICADOR`).
- **Google, solo por la API y por su puerto**, y todo lo que lo llama pasa antes por `contar`, que es el tope.
- **El precio de venta no vive en el producto**: es del plato, en la carta (M10).
- **Los vacíos** (0046): todo `EstadoVacio` lleva su dibujo, que **solo se importa con** `import()`. Lo filtrado que no está va en `NadaConEso`.
- **El acento como texto va en** `acentoParaTexto`: en claro no llega a 4,5:1.
- **Las capturas de referencia son las de Linux**, con **Ubuntu fijo**. Una pantalla que cambia a propósito se da por buena con `pnpm capturas:traer` y **mirándola**. **Con Vera no entra ninguna otra prueba.**
- **Las fotos de producto** (0046): la fila guarda claves, nunca direcciones, y los enlaces se firman **de una tanda**.
- **Almacén e Inventario** (0049): la app es **Almacén** (`/almacen`, `app.almacen`) y contar la cámara es **Inventario**. Por dentro, el acto de contar sigue siendo `recuento` (tipo de movimiento, `cerrar_recuento`, `accion.cerrar_recuento`). Las direcciones viejas llevan a las nuevas (`direccionesViejas.ts`); la historia no se reescribe.
- **Lo congelado no caduca** (0049): no sale entre lo que caduca; avisa por lo que lleva congelado (`congelado_aguanta_meses`, tres por defecto) y congelar no pregunta la caducidad. El día lo cuenta la base; qué se dice, `congelado.ts`.
- **El Tablón** (0049): escribe cualquiera del local, a su nombre; quita su autor o quien ve Equipo; la cocina no ve lo de sala ni al revés. Con hora, sale en «Hoy» y en el Calendario **leído de su tabla**, no copiado.
- **La carta subida** (0049): páginas en imagen, subidas de una en una y **publicadas de una vez**; solo claves de `cartas/<su local>/`.
- **En el iPhone** (0049): **nada se esconde con** `:has(:empty)` (`usarSinNadaDentro`; lo tasa `sin-has.prueba.ts`), y **lo pegado abajo lleva** `--desfase-abajo` y se aparta con el teclado (`anclaAbajo.ts`).
- **Las semillas se cargan por orden alfabético**: su nombre es su turno (lección 118).
- **El lector** (L): un código se busca por `codigo_de_barras` exacto; lo nuevo va al alta con él puesto; Open Food Facts **solo propone**, y solo para códigos de tienda; y un lector de mano se reconoce por su velocidad (`esDeUnLector`), fuera de los campos. La única excepción de `.dependency-cruiser.cjs` para él es su fichero WebAssembly (`?url`), que la herramienta no sabe resolver.
- **Los clientes del admin** (0050): el admin **lee** a sus clientes solo por las funciones de la `0049` (`los_clientes`, `un_cliente`, `el_uso_de`, `lo_que_hacen_los_clientes`), que devuelven lo justo y

comprueban ellas mismas que quien pregunta es admin o el reloj. **Lo que el admin toca del cliente llega a la auditoría del cliente** ( `anotar_desde_el_admin` ). **La actividad cuenta el trabajo, no montar la cuenta**, y la decide el dominio ( `laActividad` ). **El correo de acceso solo cambia con el enlace del correo nuevo**, que hay que pulsar; en la base, solo huellas.

- **Una prueba no fija el día si la base apunta con `now()`** : cuenta desde hoy (lección 120).
- **Nada del Panel se esconde por vacío** (0051): cada tarjeta dice su vacío, «Hoy» es una línea y el Tablón también. **Las mermas son el quinto destino de Almacén** ( `/almacen/mermas` ; la dirección de antes lleva allí). **Google y el punto exacto van juntos en Tu local**, y el marcado a mano manda. **Lo que no es de alguien lo devuelve al Panel diciéndoselo.**
- **La campana** (0052): **los avisos los escribe el sistema** ( `enNombreDelSistema` , que se puede anidar) en la misma transacción que lo que los provoca; cada uno solo lee los suyos y solo marca «leído» (lo cuida un disparador). **A quién le llega lo dice `estook.quien_recibe` , y solo al sistema.** Qué avisos hay, qué dicen y cómo vienen de fábrica, en el dominio ( `avisos.ts` ); qué pide cada uno, en `LO_QUE_PIDE_EL_AVISO` ; y de cada evento su aviso, en `lo-que-avisa.ts` . **Uno por cosa y persona. El correo sale con el comando ya guardado** (el despachador) y lo reintenta el reloj; **sin campana no hay correo**. «Hoy» es lo que hay que hacer; la campana, lo que ha pasado: no se mezclan. **La campana no se esconde nunca**, tampoco en el modo cocina.
- **Lo que avisa el reloj** (0053): se cuenta **a nombre de quien lo recibe** ( `comoSiFuera` , en `lo-que-avisa-el-reloj.ts` ), con sus permisos, y cada local en su punto de guardado; sus fallos no hacen repetir el día. **Como sistema, el reloj solo lee qué locales hay y pone su ficha de Google** (tres políticas de la `0051` ). Solo cuentas al día o en prueba. «Mañana toca pedir» **no prepara el pedido**: lo prepara tocarlo.
- **El pedido sugerido** (0053): el gasto se reparte por días de la semana con 14 días de historia o más ( `pesosDeLaSemana` ) y **lo mandado que no ha llegado se descuenta**; la cuenta sigue siendo una, `cuantoPedir` , para la ficha, el pedido y «Hoy».
- **Los informes** (0053): periodos cerrados, **las cifras por el camino de las tarjetas** ( `calcularElIndicador` ), las frases en el dominio ( `informes.ts` ), y el enlace del correo con la fecha ( `?del=` ). **Una cifra se escribe en un sitio**: `laCifraEscrita` .
- **La nota en Google** (0053): cada tres días y al mirarla si tiene más de uno, todo por `contar` (que vive en `nota-de-google.ts` ); cada lectura deja su día en `estook.nota_en_google` , y solo el sistema la escribe.